Beyond E2E Scripts: Using LLM-Proposed Scenarios Without Letting the LLM Be the Oracle

Alexander Kolpakov*

October 9, 2026

Abstract

Large language models can enlarge the scenario space explored by end-to-end tests, but they are unsafe as test oracles: they can invent impossible states, misread application semantics, and overstate weak evidence. Bayesilisk separates scenario proposal from verification. App-specific connectors publish a bounded search surface consisting of routes, actions, identifiers, state facts, invariants, and executable probe hooks. The Bayesilisk core expands that surface into candidate scenarios, validates their structure and declared dependencies, and verifies connector-returned evidence against explicit invariants. Grassmann-style attention guides exploration of the built-in invariant catalog; a Bayesian score orders evaluated catalog findings. The same boundary supports coding-agent assistance: agents may draft connector context, action mappings, and repair briefs, but observations must come from execution and issue readiness from verification. Reusable typed motifs provide a category-theoretic interpretation of workflow composition. A deterministic OpenAPI scanner drafts part of the search surface, and a stateless controller coordinates motif binding, verification, and repair hand-offs until a bounded stopping criterion is met.

This paper gives a compact formal model for that loop and reports a Cal.com case study at a fixed May 2026 revision. From connector-declared route facts and a typed Abstract Bayesilisk Action Graph (ABAG), Bayesilisk generated seven probes. All of them returned deterministic expected-vs-observed violations. One upstream report has a targeted fix pull request that identifies the root cause and adds a regression Playwright test; an independent human reviewer approved that fix.

1 Introduction

End-to-end suites (E2E) are reliable when they fail because their assertions are explicit, executable, and repeatable. However, their weakness is coverage. A suite usually contains the workflows a team remembered to encode under the time and productivity constraints, not the combinatorial neighborhood around stale identifiers, role changes, cancelled state, direct routes, private links, feature flags, or replayed workflow tokens.

On the other hand, LLM-driven browser agents have the opposite shape. Prior work shows that language models can interleave reasoning and external actions, but realistic web-agent benchmarks still show large reliability gaps, and hallucination remains a known failure mode in generated text and reasoning [15, 16, 11]. Such agents can suggest unusual states and longer sequences, but they are unreliable when asked to decide whether the observed behavior is correct. They also cannot guarantee reliable generation of valid scenarios. Recent agentic E2E work still asks the inherently stochastic model to infer pre/postcondition oracles [14]; `Bayesilisk` instead keeps the oracle within deterministic connector-supplied bounds.

`Bayesilisk` [1] is built around a narrow separation: models propose where to look, but deterministic rules decide what counts as a valid scenario, and what constitutes a failure. Ranking is a later triage step, not an oracle. The connector for an application exposes context and executable hooks; it does not own the scenario search strategy. The `Bayesilisk` core generates candidate scenarios from the declared surface, validates candidates against the connector contract, asks the connector to execute them, and verifies

*corresponding author; e-mail address: akolpakov@uaustin.org; postal address: University of Austin, 522 Congress Street, Austin TX, 78701 USA

returned evidence against explicit invariants. Attention guides catalog exploration, and Bayesian scores order evaluated catalog findings; connector route and sequence proposals are expanded in deterministic declaration order.

The workflow form of this boundary is an Abstract Bayesilisk Action Graph (ABAG). In an ABAG, `Bayesilisk` sees typed abstract tokens such as `resource.public_id/booking`; the connector/adaptor supplies refinements such as `booking.uid` so the abstract proposal can be executed in the concrete application.

This is close in spirit to property-based testing, where generators produce inputs and properties act as oracles [8]. The difference is the target: `Bayesilisk` treats a modern UI/API application as the system under test and uses browser/API probes, implemented in the artifact with Playwright, as evidence sensors [13]. It also treats LLM proposals as untrusted generator hints, not as properties and not as verdicts.

The practical value is that teams do not have to choose between narrow E2E scripts and unconstrained LLM agents. If a team can describe the app surface it already understands—actors, routes, actions, fixtures, identifiers, and rules—`Bayesilisk` can sweep the adjacent scenario space, prioritize suspicious combinations, and emit deterministic issue payloads. Our claim is deliberately bounded but useful: `Bayesilisk` does not replace domain knowledge; it turns that domain knowledge into a larger, more variable, and repeatable search procedure.

This paper makes five contributions:

1. a trust-boundary model for using LLM-proposed scenarios without using an LLM as the oracle;
2. a mathematical formulation of connector contracts, scenario validation, Grassmann-style attention, Bayesian ranking, and deterministic evidence verification, including typed ABAG workflow composition and a motif library with a category-theoretic interpretation of token configurations, action sequences, and connector refinements;
3. a Cal.com artifact showing seven generated probes, seven observed expected-vs-observed violations, reproducible issue payloads, and an upstream targeted fix pull request with human approval review for one reported issue;
4. a deterministic API-surface scanner and a stateless loop controller that automate scan, motif binding, verification, and repair-brief generation, with execution and repair hand-offs and a bounded stopping criterion, keeping the oracle deterministic throughout; and
5. an explicit account of scope and generalization: connectors are app-specific by design, action-sequence breadth is governed by the connector-declared graph, and the Cal.com comparison summaries illustrate the architectural separation from E2E-only, Playwright-only, weak-model, and LLM-agent workflows.

2 Claim Audit

There are five important questions to be answered about `Bayesilisk`, including its architecture and use cases.

Who generates scenarios? The connector declares the app-specific search surface: routes, actions, identifiers, state facts, invariants, and valid mutation or action schemas. For workflow probes, it declares a typed ABAG: actions, abstract token preconditions, abstract token products, state requirements, and connector refinements. `Bayesilisk` generates concrete candidate scenarios from that declared surface, optionally using LLM proposals as additional candidate ideas. The connector does not enumerate final tests.

Who executes scenarios? The connector executes validated probe requests through the real application, local fixtures, Playwright, API calls, or a test harness. It returns observed evidence such as semantic status, URL, trace, screenshot, state transition, or execution error.

Who decides pass or fail? `Bayesilisk` applies deterministic invariants to evidence returned by the connector. For status-based observations, the connector also emits a Boolean comparison, which the connector-verification interface checks against the expected and observed statuses before producing findings. The expected behavior is supplied by the contract; the LLM does not judge it.

How can coding agents participate safely? A coding agent may help collect connector requirements, draft source context, write connector code in the target repository, and propose repairs after a verified finding.

Those steps are admissible only when provenance is recorded and verifier-owned fields are kept out of agent-authored artifacts. Scenario plans cannot contain `observedStatus`, `passed`, risk scores, or issue readiness. Observed evidence is admitted only after connector execution against local fixtures. Repair briefs are derived from verified issue-ready findings, not from raw connector output.

What external validation exists? The Cal.com artifact produced issue-ready findings. One public issue, #29399 [2], has an upstream contributor fix pull request, #29400 [5], that identifies a missing null guard in `processReschedule` and adds a regression Playwright test. The fix PR has a human approval review that calls the null guard correct and the test coverage solid [7]. A separate workflow issue thread, #29407 [3], remains open at capture time.

3 Formal Model

Let an application connector publish a contract

$$C = (R, A, F, I, M, G),$$

where R is a set of route patterns, A is a set of connector actions, F is a set of observable state facts and identifiers, I is a set of deterministic invariants, M is a set of parameter mutation schemas, and G is an Abstract Bayesilisk Action Graph. The connector is app-specific; the `Bayesilisk` core is a generic function over C. In this sense the connector is an application adaptor: it translates between concrete app fixtures and the abstract contract consumed by `Bayesilisk`.

3.1 Scenario Generation and Validation

From connector-declared route facts and mutation schemas, `Bayesilisk` constructs a route-mutation candidate of the form

$$s = (r, a, i, p, m),$$

where $r \in R$ is a declared route pattern, $a \in A$ is a declared connector action, $i \in I$ is a declared invariant, p is a declared route parameter, and $m \in M(p)$ is an allowed mutation for that parameter, such as unknown, stale, foreign, missing, or duplicate. The connector does not enumerate this final candidate. It declares the searchable surface and the allowed mutation schema; `Bayesilisk` performs the expansion, deduplication, and structural validation.

3.2 Abstract Bayesilisk Action Graphs

For longer workflows, `Bayesilisk` uses a typed Abstract Bayesilisk Action Graph (ABAG)

$$G = (A, T, \text{req}, \text{prod}, \text{bind}),$$

where A is the connector action set and T is a set of typed tokens. A token $\tau \in T$ has three relevant fields:

$$\tau = (u, c, \rho).$$

The components deliberately separate the portable abstraction from the app-specific adaptor vocabulary:

- u is the universal token, for example `resource.public_id` or `state.cancelled`;
- c is an optional class that narrows the token, for example `booking`, `event_type`, or `calcom_user`;
- ρ is an optional concrete connector refinement, for example `booking.uid` or `user.username`.

The abstract key used by the `Bayesilisk` core is

$$\kappa(\tau) = (u, c).$$

The refinement ρ is not used to satisfy graph dependencies. It is connector-facing metadata that lets the adaptor execute a generated abstract probe against concrete fixtures. An unqualified requirement (c omitted)

can match any resource class with token u; a qualified requirement must also match c. Operationally, let $K(\tau) = \{u, (u, c)\}$ for a qualified token and $K(\tau) = \{u\}$ otherwise. This models the implementation's qualified and unqualified dependency keys.

Each action $a \in A$ has required and produced token sets, $\mathrm{req}(a) \subseteq T$ and $\mathrm{prod}(a) \subseteq T$. A sequence

$$\sigma = (a_1, \ldots, a_k)$$

is valid for a terminal goal action a_k only if

$$\forall t \in \{1, \ldots, k\} : \bigcup_{\tau \in \mathrm{req}(a_t)} K(\tau) \subseteq \bigcup_{j<t} \bigcup_{\tau \in \mathrm{prod}(a_j)} K(\tau), \qquad k \leq d_{\max}.$$

Goal parameter bindings are also typed. If a terminal action has a parameter p, then $\mathrm{bind}(a_k, p) = \tau$ means that `Bayesilisk` may bind p to previously produced tokens satisfying $K(\tau)$. If ρ is present, `Bayesilisk` emits it in the generated proposal so the connector can look up the concrete value. Thus `Bayesilisk` may reason over `resource.public_id/booking`, while the Cal.com connector executes with `booking.uid`. This is the central refinement boundary: Bayesilisk composes and validates over abstract tokens; the connector/adaptor executes concrete app operations through refinements. The current sequence builder selects the first declared producer for each missing dependency and topologically orders the selected actions. It emits at most one sequence per rule, subject to depth and proposal-count limits; it does not enumerate all valid paths through G.

This makes the reusable unit a graph motif rather than an app noun. The same abstract sequence

`resource.create` → `state.transition(revoked)` → `identifier.replay(boundary)`

can describe cancelled booking replay, voided invoice link replay, expired password-reset token replay, or revoked invitation acceptance. Each app needs its own connector refinements and execution hooks, but the `Bayesilisk` core sees the same class of verifier-search problem.

Validation is deterministic:

$$V_C(s) = \begin{cases} 1 & \text{if } s \text{ satisfies the structural and dependency constraints of } C, \\ 0 & \text{otherwise.} \end{cases}$$

This gate checks declared routes, actions, parameters, mutation rules, and token dependencies as applicable to each proposal path. Model proposals are also subject to deterministic validation: unknown target planes or ids, duplicate proposals, non-string fields, malformed JSON, and impossible counts become proposer diagnostics. Connector sequence rules whose dependencies cannot be constructed within the bound produce no proposal. These checks constrain candidate structure; execution must still establish that the concrete fixture and action mappings work.

3.3 Motif Library and Categorical Interpretation

A *motif* is a reusable typed template for a parameter mutation or a workflow obligation, together with an expected status. A categorical interpretation can be given to the ABAG dependency model [12]: objects are configurations S of available abstract dependency keys, and an enabled action induces a transition

$$S \xrightarrow{a} S \cup \bigcup_{\tau \in \mathrm{prod}(a)} K(\tau), \qquad \bigcup_{\tau \in \mathrm{req}(a)} K(\tau) \subseteq S.$$

Paths through these transitions compose by concatenation, with empty paths as identities. The depth bound restricts the search over paths, not the underlying category. This is a model of accumulated dependency facts; execution determines whether the concrete application satisfies the terminal obligation. For example, a create–cancel–replay motif requires rejection of the replay. Connector refinements provide an intended interpretation in concrete fixture operations. The implementation checks token dependencies and observed outcomes; it does not mechanically establish functor laws or connector correctness.

Motifs are grouped into versioned packs. The binder matches parameter kinds or abstract token names and emits derived route facts or ABAG sequence rules. Generated proposals then enter the existing verification workflow. Workflow binding still requires a connector-declared action graph, and concrete parameter names and fixture values require application-specific adaptation. The core pack includes unknown, foreign, and wrong-parent identifiers, cancelled-state replay, superseded tokens, and missing sub-resource references. The retained Cal.com evidence below directly supports unknown/stale identifier handling and cancelled-booking replay; it does not validate every motif in the pack. Packs may be distributed as an open core or as optional signed, licensed packs; this affects motif availability, not the verification rule.

3.4 Grassmann Attention

The attention layer is a prioritizer, not an oracle. Conceptually, invariant descriptions and observations define local subspaces in a feature space. For an invariant plane P_i and context vector q, a Grassmannian proximity score can be written as

$$\gamma_i(q) = \|\Pi_{P_i} q\|_2^2,$$

where Π_{P_i} projects q onto the invariant plane. This follows the standard geometric view of subspaces on Grassmann manifolds [9].

However, $\gamma_i(q)$ is only an ideal geometric score. The current implementation does not compute this projection directly. Instead, it uses an interpretable bounded anchor-plane proxy, α_i, whose novelty term can include embedding similarity to the same invariant-plane anchors. Thus α_i is the operational attention score used for ranking planes in the current artifact, while γ_i describes the intended Grassmannian interpretation. For each invariant i, `Bayesilisk` computes

$$\alpha_i = \mathrm{clip}_{[0,1]}\left(w_f f_i + w_u u_i + w_s \sigma_i + w_p p_i + w_n n_i - d_i\right),$$

with current weights

$$(w_f, w_u, w_s, w_p, w_n) = (0.45, 0.25, 0.15, 0.10, 0.05).$$

The variables are computed directly from context facts. The notation is: P_i is the invariant plane; q is the context vector; Π_{P_i} is projection onto P_i; $\gamma_i(q)$ is ideal Grassmann proximity; α_i is the operational attention score; T_i is the number of facts whose `invariantId` is i; F_i is the number of those facts that fail; N_i^{PW} is the number of those facts whose source is Playwright; K_i is the keyword-hit count; E_i is embedding similarity; σ_i is configured invariant sensitivity; and d_i is fixed/muted decay. The normalized feature names are: f_i for failure density, u_i for untestedness, p_i for Playwright evidence, and n_i for novelty.

The current artifact uses three saturation thresholds:

$$\tau_T = 4, \qquad \tau_P = 3, \qquad \tau_K = 6.$$

They are calibration constants, not oracle rules. τ_T means that after four observations a plane is no longer treated as materially untested; τ_P means that three Playwright observations saturate the browser-evidence term; and τ_K means that six keyword hits saturate lexical novelty. These values are current implementation defaults chosen to make attention saturate quickly on small connector artifacts; they should be treated as tunable ranking parameters and ablated in larger evaluations. Changing these thresholds changes prioritization pressure only; it does not change candidate validity or pass/fail verification. Then

$$f_i = \begin{cases} F_i/T_i & T_i > 0, \\ 0 & T_i = 0, \end{cases} \qquad u_i = \begin{cases} 1 & T_i = 0, \\ \max(0, 1 - \min(T_i, \tau_T)/\tau_T) & T_i > 0, \end{cases} \qquad p_i = \min(1, N_i^{PW}/\tau_P).$$

The current attention loop evaluates the built-in invariant catalog. The sensitivity σ_i is read from the `INVARIANT_SENSITIVITY` table maintained by the `Bayesilisk` authors in the core configuration. It encodes how sensitive an invariant plane is considered for search prioritization. A catalog invariant absent from that table receives the neutral default 0.5. External connector invariant ids are not added to the attention planes. The novelty term is

$$n_i = \max(\min(1, K_i/\tau_K), E_i),$$

where K_i is the context keyword-hit count for invariant i, and $E_i \in [0,1]$ is optional embedding similarity to the invariant anchor plane (0 if embeddings are disabled or unavailable). The decay d_i is 0.12 for fixed or regression-watch invariant ids, 0.20 for muted invariant ids, and 0 otherwise, using the larger value if both apply. The largest α_i values select planes for exploration. They do not determine whether a behavior is correct.

3.5 Bayesian Ranking

Bayesian ranking runs only after a deterministic invariant result exists. It is used to order findings in the report, assign posterior-mode labels, and decide which issue candidates or probes appear first. It is not used to create a failure. Operationally, the pipeline is: `Bayesilisk` generates or accepts a candidate scenario; contract validation rejects malformed or non-applicable candidates; the connector executes valid probes or returns observed evidence; a deterministic invariant produces pass or fail; and only then does `Bayesilisk` compute ρ_i. Thus Bayesian ranking answers which verified finding a tester should inspect first. It does not answer whether the behavior is a bug.

For catalog and generated verifier scenarios, the ranking inputs come from two places: invariant metadata and connector/context observations. In the current implementation the invariant metadata is the `Invariant` object:

$$I_i = (\texttt{id}, \texttt{expected}, \pi_i, L_i^{fail}, L_i^{pass}, \texttt{difficulty}, \phi_i).$$

The `Bayesilisk`-authored catalog stores these objects in `bayesilisk/invariants.py`; report output serializes the same likelihood metadata as `failLikelihood` and `passLikelihood`. Thus π_i, L_i^{fail}, and L_i^{pass} are not inferred during a run. They are prior/ranking constants supplied by the invariant author with the invariant definition; they are not generated by the LLM and are not derived from connector execution. External connector observations use the separate status-comparison and fixed-priority path described below.

The run-time context adjustment is computed from observation tags and explicit context fields. For finding fingerprint x, invariant i, and scenario s, the implementation computes

$$\Delta_i = -0.28\,\mathbf{1}_{x \in Fixed} + 0.18\,\mathbf{1}_{x \in Confirmed} - 0.45\,\mathbf{1}_{x \in Muted} + A_i + B_s,$$

where x is the finding fingerprint, A_i is a context adjustment for invariant i, and B_s is an explicit context adjustment for scenario s. The sets $Fixed$, $Confirmed$, and $Muted$ are populated from `fixedFingerprints`, `confirmedFingerprints`, and `mutedFingerprints` in the context/observation file, plus fingerprints extracted from context text, which are treated as muted. In the contextual report path, A_i defaults to $\min(0.18, 0.018K_i)$, using context keyword hits, and a numeric entry in `priorAdjustments` overrides that value. B_s is read from `scenarioAdjustments` and defaults to zero.

The likelihood used in the posterior is selected after deterministic verification. For a concrete scenario s, `Bayesilisk` merges its fragments into facts F_s and calls the invariant evaluator:

$$\phi_i(F_s) \to (b_i, o_i),$$

where b_i is the Boolean evaluator result and o_i is the evaluator's observation text. The reported deterministic result is

$$r_i = \begin{cases} \text{pass} & \text{if } b_i = \texttt{true}, \\ \text{fail} & \text{if } b_i = \texttt{false}. \end{cases}$$

Only then does `Bayesilisk` choose the scalar likelihood used for ranking:

$$\ell_i = \begin{cases} L_i^{pass} & \text{if } r_i = \text{pass}, \\ L_i^{fail} & \text{if } r_i = \text{fail}. \end{cases}$$

So "computed" here means deterministic branch selection from invariant metadata after the evaluator has already returned pass/fail. There is no learned likelihood estimator in the current artifact, and the likelihood does not decide whether the scenario failed. It only changes the ordering of already evaluated findings. The adjusted prior is

$$\tilde{\pi}_i = \text{clip}_{[0.05,0.95]}(\pi_i + \Delta_i).$$

The posterior risk score is

$$\rho_i = \frac{\tilde{\pi}_i \ell_i}{\tilde{\pi}_i \ell_i + (1 - \tilde{\pi}_i)(1 - \ell_i)}.$$

This is a ranking score in the Bayesian sense [4]. The report sorts by decreasing ρ_i, then by posterior mode and finding id, so ρ_i controls report order, follow-up probe order, and triage labels. Pass/fail is produced only by the invariant evaluator.

Direct connector-observed findings use the same general report pipeline but a different scoring path. If connector evidence already contains a deterministic mismatch between expected and observed status, the finding is evidence-bearing before Bayesian ranking. `Bayesilisk` records it as a context-observed breakage, marks it issue-ready, and assigns a fixed risk score of 0.99 for report priority. This value is not a calibrated probability of a real application defect. This is evidence-first ranking, not learned Bayesian inference. The path is connector-generic; Cal.com is the current case study using it, not a special rule in the core.

3.6 Evidence Verification

Execution is a connector function

$$X_C(s) \to e,$$

where e is observed evidence. For the Cal.com connector, the core evidence fields are `expectedStatus`, `observedStatus`, route, target URL, invariant id, and source context. The verifier applies

$$\phi_i(e) = \begin{cases} \mathsf{invalid} & \text{if required evidence is absent or malformed,} \\ \mathsf{fail} & \text{if valid } e \text{ has } e.\texttt{observedStatus} \neq e.\texttt{expectedStatus}, \\ \mathsf{pass} & \text{otherwise.} \end{cases}$$

Only fail with evidence becomes an issue-ready finding. invalid is a connector/proposer diagnostic. In the connector-verification interface, the required observation fields include integer statuses, a Boolean `passed` equal to their comparison, execution-source metadata, and artifact references. These checks establish structural consistency; they do not independently attest that an execution occurred or that its expected status is the correct application specification.

4 Implementation Boundary

`Bayesilisk` implements separate ledgers for provenance, attention, proposal, execution evidence, and verification. The provenance ledger records which repository, source claims, fixture boundary, and connector contract produced a run. The attention ledger records which invariant planes should be explored. The model-proposal ledger records accepted and rejected model proposals. The execution-evidence ledger records connector-returned observations. The verification ledger records deterministic invariant outcomes, fingerprints, issue readiness, and risk scores. Only the verification ledger contains issue-ready pass/fail results.

This boundary matters for connector design. A connector should expose a small vocabulary of actions and facts. It should not filter away uncomfortable cases or silently reinterpret a 200 response as acceptable when the contract requires rejection. The verifier applies the ruleset supplied by context and schema. Application-specific expectations and fixture mappings remain the connector author's responsibility.

The agent-facing workflow is therefore a packaging of the same proof boundary, not a new oracle. A coding agent can interview for connector requirements, record provenance, produce a connector prompt packet, propose a scenario plan, and draft a fix brief. The verifier accepts those artifacts only as context or candidate input. Concrete observations must still come from connector execution, and fix briefs must still trace to verified issue-ready findings.

4.1 Surface Scanning and the Automated Loop

`Bayesilisk` provides a deterministic OpenAPI scanner for drafting route facts. The current implementation handles GET, POST, PUT, PATCH, and DELETE operations with inline path or query parameters. Parameter names and schema hints suggest identifier, tenant, and token kinds. The scanner does not resolve `$ref`

references, inspect request bodies, infer authentication fixtures, or construct an action graph. Motif binding supplies candidate mutation rules and expected statuses; connector authors must check their applicability and implement the concrete actions.

The stateless loop controller accepts this draft or a supplied source context, binds motifs, returns source-validation diagnostics and proposals, and requests connector execution. On later calls it validates observations, builds reports, accumulates issue-ready fingerprints, and produces repair briefs. Connector implementation, execution, and application of repairs remain external steps. The caller persists the returned state between calls. Invalid observations block a round; accepted rounds stop when K consecutive rounds add no new issue-ready fingerprint (default $K = 2$) or the round limit is reached (default six). The phase name `converged` denotes this stopping rule: repeated evidence for an unresolved finding can satisfy it, so it does not establish that all defects have been repaired. The controller coordinates the workflow while `Bayesilisk` performs deterministic verification of submitted evidence.

5 Cal.com Case Study

We evaluated the current connector architecture on a local Cal.com checkout [6]. The tested repository and revision were:

- Repository: `https://github.com/calcom/cal.com`
- Commit: `180ede28f0bddf2738933a6e60a8e80f6116d7da`
- Commit date: `2026-05-14 19:30:21 +0000`
- Commit subject: font-stack fallback fix for non-Latin script support (`#29346`)
- Bayesilisk seed: `150`

The connector supplied three route facts for booking pages with `rescheduleUid` behavior and one typed ABAG for a longer booking workflow. `Bayesilisk` generated six route-mutation probes and one workflow-sequence probe. The sequence was:

$$\texttt{create-booking} \rightarrow \texttt{cancel-booking} \rightarrow \texttt{open-public-booking-route}.$$

The ABAG was not Cal.com logic embedded in the `Bayesilisk` core. The checked-in connector acted as an adaptor. Its `create-booking` action produced four abstract-to-concrete refinements:

- `resource.public_id/booking` $\mapsto$ `booking.uid`;
- `resource.id/booking` $\mapsto$ `booking.id`;
- `resource.public_id/event_type` $\mapsto$ `eventType.slug`;
- `principal.actor/calcom_user` $\mapsto$ `user.username`.

The `cancel-booking` action required `resource.id/booking` and produced `state.cancelled/booking`. The `open-public-booking-route` action bound `rescheduleUid` to `resource.public_id/booking`. `Bayesilisk` matched only the abstract keys and generated the sequence. The Cal.com Playwright connector used the `refines` names to execute real fixture operations and return observed status evidence.

Table 1 lists the retained artifacts for this case study, from connector inputs through execution evidence and issue payloads.

Table 1: Checked-in Cal.com artifact set.

Artifact	Path
Connector code	`examples/calcom/bayesilisk-probes.e2e.ts`
Route context	`examples/calcom/source-context.json`
Generated route proposals	`examples/calcom/generated-proposals.json`
Sequence context	`examples/calcom/sequence-source-context.json`
Generated sequence proposal	`examples/calcom/generated-sequence-proposals.json`
Execution evidence	`examples/calcom/execution-context.json`
Bayesilisk report	`examples/calcom/reports/report.json`
Issue payloads	`examples/calcom/reports/issue-payloads.json`
Upstream outcomes	`examples/calcom/upstream-outcomes.md`

Table 2 gives the expected and observed semantic statuses for all seven executed probes.

Table 2: Verified Cal.com connector observations (semantic statuses).

Area	Mutation	Expected	Observed
Public booking page	unknown id	404	200
Public booking page	stale id	404	200
Private booking link	unknown id	404	200
Private booking link	stale id	404	200
Dynamic booking page	unknown id	404	200
Dynamic booking page	stale id	404	200
Cancelled booking replay	cancelled uid	409	200

All seven probes executed against local Playwright fixtures without unexpected Playwright test failures [13]. All seven returned evidence that violated the connector-declared invariant. The route mutations expected semantic status `404` and observed `200`. The cancelled-booking replay expected semantic status `409` and observed `200`. `Bayesilisk` classified the findings as `breakage.context-observed` and produced issue-ready payloads. Unknown-identifier handling and cancelled-state replay also appear in the core motif pack with provenance notes referring to this example. The other motif families are broader library coverage, not additional findings in this seven-probe artifact. The retained proposals were generated directly from connector rules and the ABAG; they do not measure the incremental benefit of the later scanner, motif binder, or loop controller.

6 Comparison Runs

Table 3 summarizes local exploratory comparisons recorded in `evidence_list.md`. The retained raw Playwright logs in `examples/calcom/baselines/` cover the six generated route probes and the one generated sequence. The nearby E2E, manual-probe, weak-model, and LLM-agent counts are historical summaries whose raw logs are not retained in the current artifact. They provide qualitative context, not a reproducible comparative benchmark.

Table 3: Cal.com exploratory comparison summary.

Workflow	Observation
Nearby E2E	14 nearby specs ran without unexpected failures; the existing suite did not cover the generated stale/unknown states.
Playwright-only probing	11 manual observations produced 4 failures; manual probes exposed only a subset of the bug family.
Bayesilisk route proposals	6 generated route proposals produced 6 findings across declared route variants.
Bayesilisk sequence proposal	1 generated workflow sequence produced 1 cancelled-booking replay finding.
Weak local model proposals	3 raw proposals produced 0 accepted probes because verifier validation rejected malformed candidates.
LLM-agent browser run	10 trials produced 3 oracle mismatches before deterministic verification.

These summaries motivate the architectural separation. The retained generated probes show how `Bayesilisk` expands declared route facts and action graphs into concrete cases. The weak-model summary records malformed proposals rejected by validation, and the LLM-agent summary records oracle mismatches. Establishing comparative effectiveness or model reliability would require retained baseline runs, matched execution budgets, and a larger evaluation.

7 Upstream Validation

Two upstream issue threads were opened from the Cal.com evidence. The first, #29399 [2], reported that an unknown `rescheduleUid` opened a normal booking flow instead of returning 404. An upstream contributor opened PR #29400 [5]. The PR identifies the root cause as a missing null guard in `processReschedule`, changes the server-side behavior to return `notFound`, and adds a Playwright regression test. The PR was then approved by another human reviewer, who characterized the null guard as correct and the regression coverage as solid [7]. The private-link variant was added to the same issue thread to avoid duplicate reporting. A separate issue, #29407 [3], reports the cancelled booking UID replay sequence and remains open at capture time.

Table 4 maps the reported finding groups to their recorded upstream responses.

Table 4: Upstream response to Bayesilisk-generated findings at capture time.

Finding group	Upstream response
Public unknown/stale `rescheduleUid`	Issue #29399 with targeted fix PR #29400: null guard, regression test, and human approval review
Private-link `rescheduleUid`	Added to issue #29399 as a related comment to avoid duplicate issue noise
Cancelled booking UID replay	Separate state-transition report in issue #29407; open at capture time

This validation is practical rather than statistical. A fix PR does not prove the entire methodology, but it does show that at least one generated finding was actionable enough for an upstream contributor to diagnose and patch, and for another human reviewer to approve the fix.

8 Scope and Generalization

`Bayesilisk` is intended to be adopted through app-specific connectors and app-agnostic verification. The following points describe that engineering boundary and the evidence needed to broaden the empirical case; they are not exceptions to the architecture.

Application scope. The current public practical validation is Cal.com. The result does not depend on Cal.com-specific verifier code: the core consumes a connector contract and generated probes through the same app-agnostic interfaces used elsewhere in `Bayesilisk`. Additional application artifacts would strengthen empirical breadth, but the required adaptation is the expected one: each app team writes a connector that exposes its own routes, actions, state facts, fixtures, and invariants.

Connector specificity. Connector-specific code is not a weakness of the architecture; it is the integration boundary. A team that knows the app specification can declare actors, routes, action hooks, identifiers, and invariants without modifying the `Bayesilisk` core. Connector quality still matters: the broader and more accurate the declared surface, the broader the generated probe space.

Agent-assisted adoption. The intended adoption path is compatible with coding agents, but not dependent on trusting them. An agent can be guided to read existing tests and route handlers, write connector source context, and implement action mappings in the target repository. The contract prevents that agent from also becoming the oracle: generated plans are checked for verifier-owned fields, provenance is recorded before execution, and any repair brief is downstream of deterministic verification.

ABAG refinement boundary. Typed ABAG tokens are the portability layer. The `Bayesilisk` core matches `token` and optional `resourceType`. It does not treat `refines` as evidence that a dependency is satisfied. `refines` is deliberately adaptor-local: it carries concrete names that the connector needs to execute a probe, such as fixture fields, URL parameters, database ids, or session handles. This keeps Cal.com names out of the verifier while still letting the generated proposal be executable.

Bounded workflow generation. Sequence generation is governed by the connector-declared action graph, sequence rules, depth bound, and proposal limit. The current builder greedily selects producers without backtracking. Additional actions and rules can expose more workflows, but the builder offers no completeness guarantee over the valid paths through the graph.

Oracle scope. The deterministic oracle is only as good as the supplied ruleset. `Bayesilisk` prevents the LLM from being the oracle; it does not remove the need for correct invariants or trustworthy execution evidence. Motif-derived expected statuses must be checked against the application's specification. The current status verifier compares one expected integer with one observed integer; alternative statuses recorded in motif metadata are not automatically accepted.

Baseline interpretation. The retained Cal.com artifact establishes seven generated, executed probes and their status mismatches at one fixed application revision. The other comparison rows are summary-level evidence. A multi-app study with retained baselines and ablations is needed to quantify coverage

gains, false-positive rates, ranking benefits, and model-dependent effects. The seven probes were generated deterministically, so they do not establish an incremental coverage benefit from LLM proposals.

9 Future Work

Two next steps would make the ranking layer more mathematically grounded while preserving the same trust boundary: attention and probability may guide search, but they must not become the oracle.

Actual Grassmann attention. The current implementation uses α_i, a bounded anchor-plane proxy for the ideal Grassmannian score $\gamma_i(q)$. A fuller version should construct an explicit feature vector q for each connector context and an orthonormal basis U_i for each invariant subspace P_i. The operational attention score could then be the projection energy

$$\gamma_i(q) = \|U_i^\top q\|_2^2,$$

or a related Grassmann distance based on principal angles between the context subspace and invariant subspace. The basis vectors can be built from invariant text, connector-declared routes/actions, nearby tests, execution evidence, and validated historical findings. This would replace the current hand-weighted proxy as the main exploration router, while leaving deterministic verification unchanged [9].

Calibrated likelihoods. The current artifact uses invariant-level ranking constants L_i^{fail} and L_i^{pass}. A stronger version should estimate likelihoods from validation data. Let Z_i mean that invariant i is genuinely violated, and let $r_i \in \{\mathsf{pass}, \mathsf{fail}\}$ be the deterministic verifier result. For a failing result, the calibrated posterior should use

$$P(Z_i \mid r_i = \mathsf{fail}) = \frac{P(Z_i)P(r_i = \mathsf{fail} \mid Z_i)}{P(Z_i)P(r_i = \mathsf{fail} \mid Z_i) + P(\neg Z_i)P(r_i = \mathsf{fail} \mid \neg Z_i)}.$$

The terms $P(r_i = \mathsf{fail} \mid Z_i)$ and $P(r_i = \mathsf{fail} \mid \neg Z_i)$ are the true-positive and false-positive behavior of that verifier/probe family. They can be estimated from historical Bayesilisk findings, seeded faults, negative controls, connector evidence quality, invariant class, and human validation signals such as upstream issue acceptance, fix pull requests, and regression tests. Passing controls should be calibrated analogously with $P(r_i = \mathsf{pass} \mid Z_i)$ and $P(r_i = \mathsf{pass} \mid \neg Z_i)$. This would turn the current hand-calibrated ranking constants into empirical reliability estimates without changing the rule that pass/fail is decided only by deterministic invariants [4, 10].

10 Conclusion

`Bayesilisk` treats LLMs and coding agents as scenario proposers, not as test oracles. The mathematical boundary is simple: candidate generation and attention may be probabilistic or heuristic, but validation and verification are deterministic functions over a connector-declared contract and connector-returned evidence. The Cal.com case study demonstrates that this separation can produce useful real-app findings: seven generated probes, seven observed invariant violations, issue-ready payloads, and a review-approved upstream fix PR for one reported issue. The adoption path is incremental: write the connector around existing app fixtures, or have a coding agent draft it under recorded provenance, let `Bayesilisk` expand the scenario space, and keep the oracle deterministic. Packaging the surface as a typed motif library, a deterministic scanner, and a stateless loop controller supports reuse across apps and coordinates bounded scan–verify–repair rounds with external execution and repair. The next research step is to repeat this artifact discipline across additional applications while keeping the `Bayesilisk` core app-agnostic.

Funding

No funding was received for this research or the preparation of this manuscript.

References

[1] Bayesilisk contributors. Bayesilisk: Beyond e2e scripts. `https://github.com/sashakolpakov/bayesilisk`, 2026. Software and artifacts.

[2] Bayesilisk contributors. Cal.com issue 29399: Booking page opens instead of 404 for missing rescheduleuid. `https://github.com/calcom/cal.diy/issues/29399`, 2026. Issue report.

[3] Bayesilisk contributors. Cal.com issue 29407: Cancelled booking uid replay opens public booking route. `https://github.com/calcom/cal.diy/issues/29407`, 2026. Issue report.

[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.

[5] Cal.com contributor. Cal.com pull request 29400: Return 404 for unknown rescheduleuid instead of opening booking flow. `https://github.com/calcom/cal.diy/pull/29400`, 2026. Human-authored fix pull request.

[6] Cal.com contributors. Cal.com. `https://github.com/calcom/cal.com`, 2026. Application under test.

[7] Cal.com reviewer. Cal.com pull request review 4321967236: Approving pull request 29400. `https://github.com/calcom/cal.diy/pull/29400#pullrequestreview-4321967236`, 2026. Human approval review.

[8] Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279. ACM, 2000.

[9] Alan Edelman, Tomás A. Arias, and Steven T. Smith. The geometry of algorithms with orthogonality constraints. *SIAM Journal on Matrix Analysis and Applications*, 20(2):303–353, 1998.

[10] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.

[11] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

[12] Saunders Mac Lane. *Categories for the Working Mathematician*. Graduate Texts in Mathematics. Springer, 1978.

[13] Microsoft. Playwright. `https://playwright.dev/`, 2026. Browser automation framework.

[14] Xiwen Teoh, Yun Lin, Duc-Minh Nguyen, Ruofei Ren, Wenjie Zhang, and Jin Song Dong. WebTest-Pilot: Agentic end-to-end web testing against natural language specification by inferring oracles with symbolized gui elements. `https://arxiv.org/abs/2602.11724`, 2026.

[15] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[16] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *NeurIPS 2023 Workshop on Agent Learning in Open-Endedness*, 2023. `https://neurips.cc/virtual/2023/83780`.